

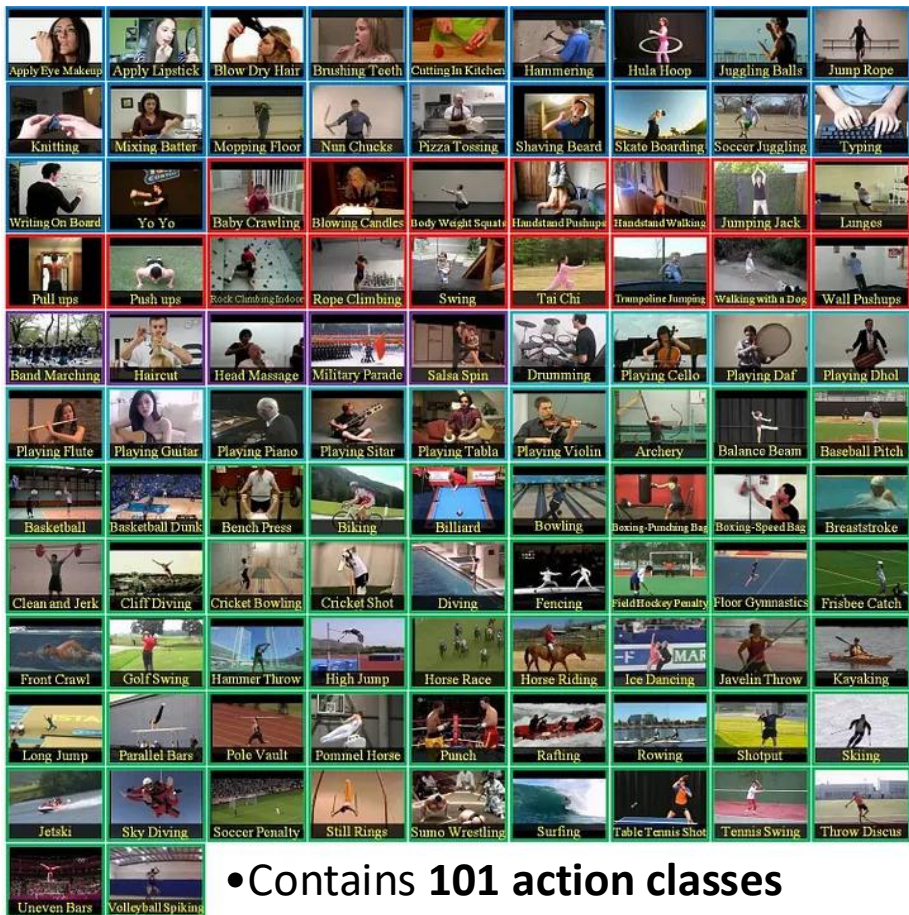
# Computer Vision and Deep Learning

Lab 09: Action Recognition

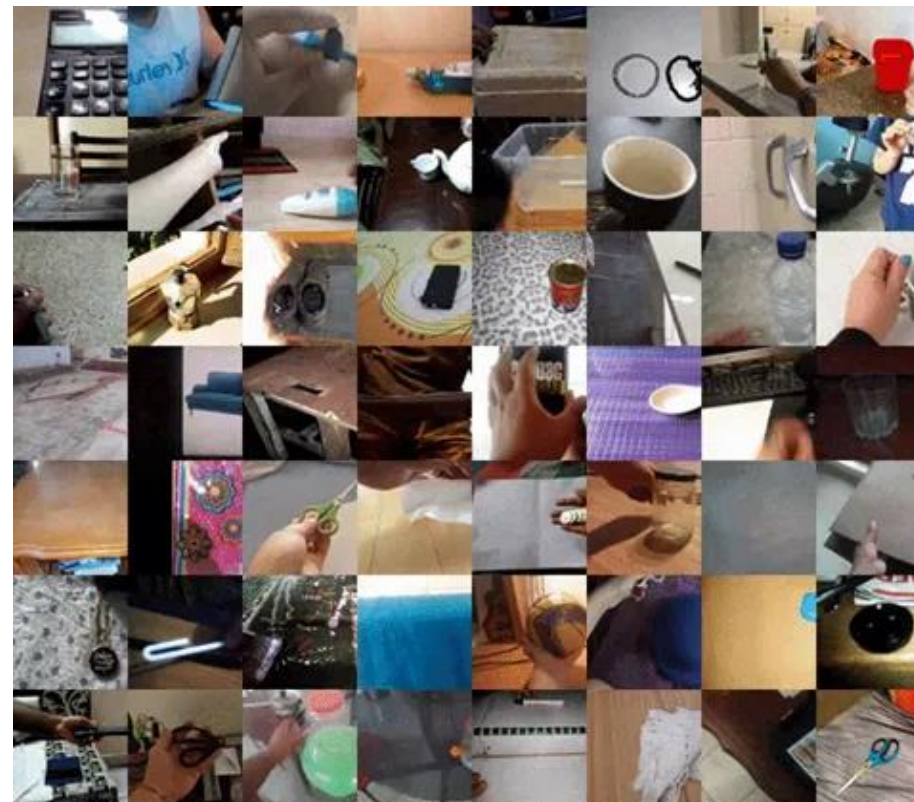
# What is an action?

- An *action* is an *event* that can be marked *on a video* with the *a priori knowledge* that it can happen there
- Usually, in practice (99% of the time), *action recognition is used for some deterministic process* (i.e., if action X happens → do Y)
- To determine that some event has occurred that needs to be counted/accounted for/controlled

# The most popular datasets



- Contains **101 action classes**
- Examples: *playing guitar, push-ups, diving*



- Contains **174 actions**, like: *“Putting something on something”* and *“Moving something up”*

# HMDB: a large human motion database

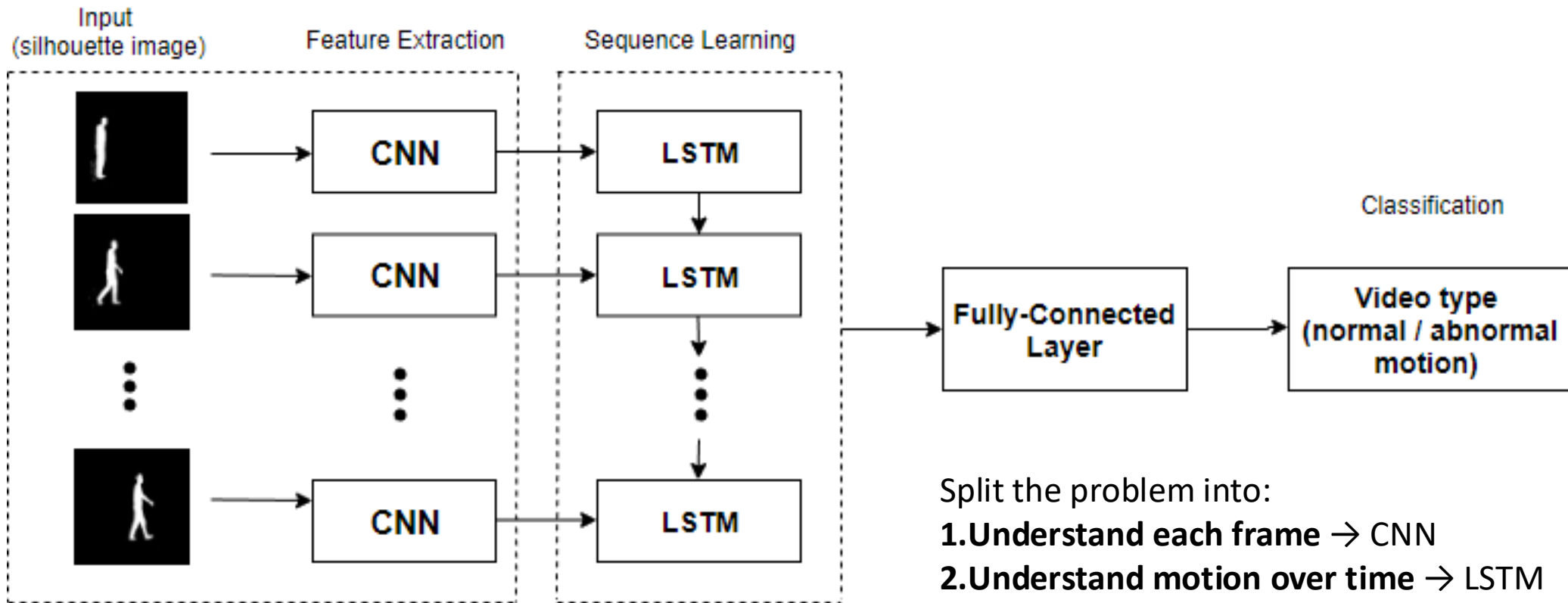
- The dataset contains **6849 clips** divided into **51 action categories**, each containing a **minimum of 101 clips**. The actions categories can be grouped in five types:
  - *General facial actions*, e.g., smile, laugh, chew, talk
  - *Facial actions with object manipulation*, e.g., smoke, eat, drink
  - *General body movements*, e.g., cartwheel, clap hands, climb, climb stairs, dive, fall on the floor, backhand flip
  - *Body movements with object interaction*, e.g., brush hair, catch, draw sword, dribble, golf, hit something, kick ball, pick
  - *Body movements for human interaction*, e.g., fencing, hug, kick someone, kiss, punch, shake hands, sword fight



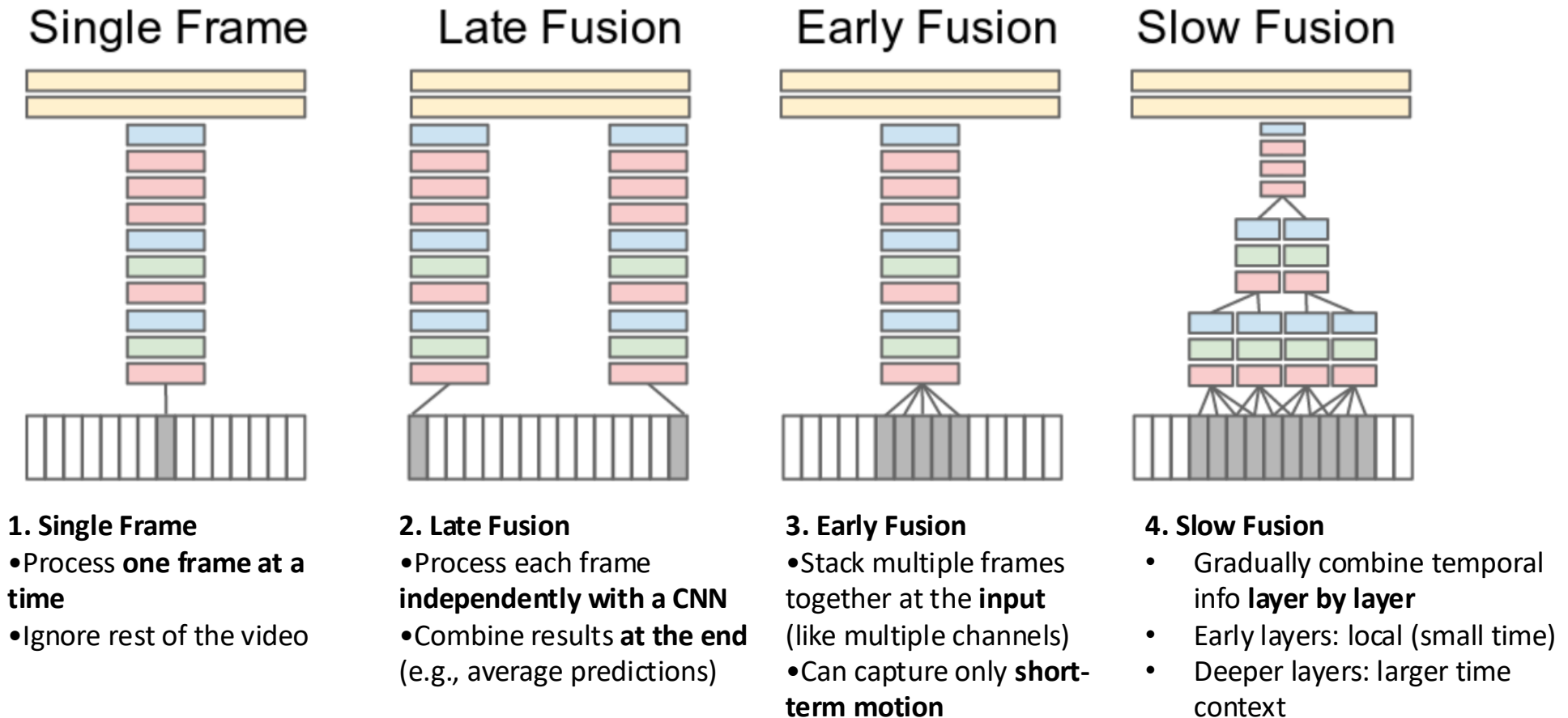
# HMDB: a large human motion database (cont'd)



# Video Recognition: LSTM + CNN



# Large Video Classification with CNN



# Two-Stream CNN

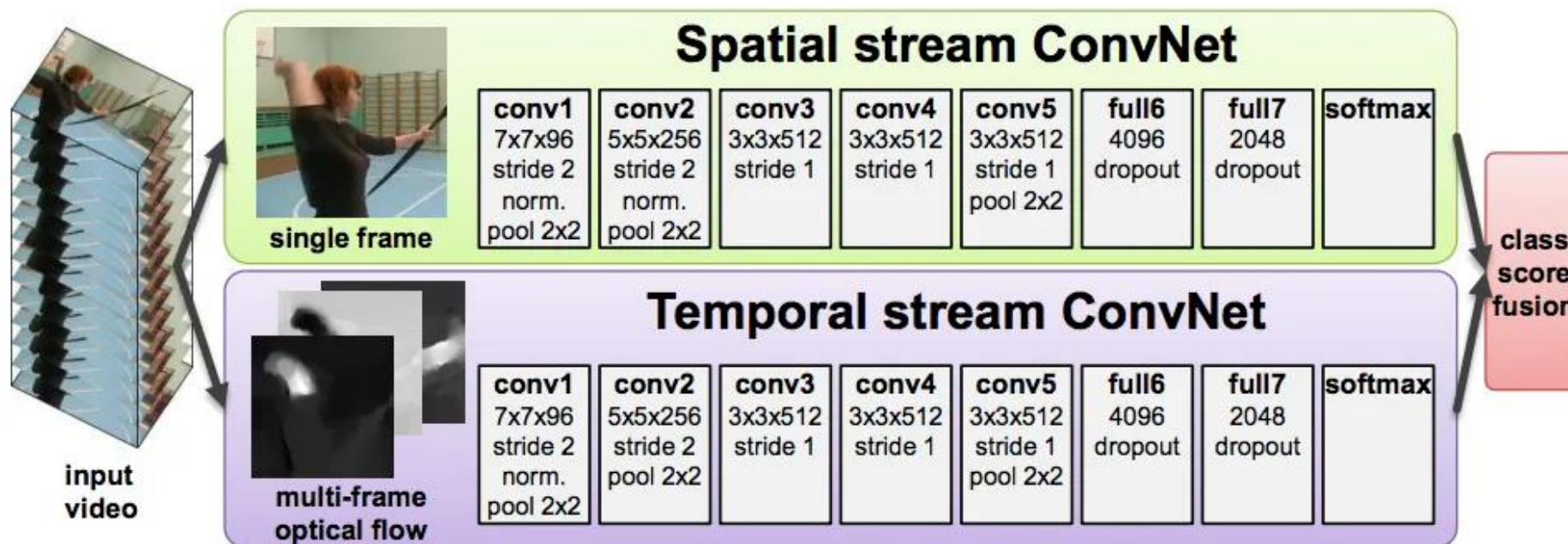
## 1. Spatial Stream

- Input: **single RGB frames**
- Processes normal images (like a standard CNN)

Learns:

- objects (person, ball, etc.)
- scene context
- body pose

Good for: *“what is in the scene”*



## Why two streams?

A single model struggles to capture both:

- **Appearance** (objects, people, scene)
- **Motion** (movement over time)

So we use **two parallel networks**, each specialized.

## 2. Temporal Stream

- Input: **optical flow (motion between frames)**
- Optical flow shows direction + speed of movement

Learns:

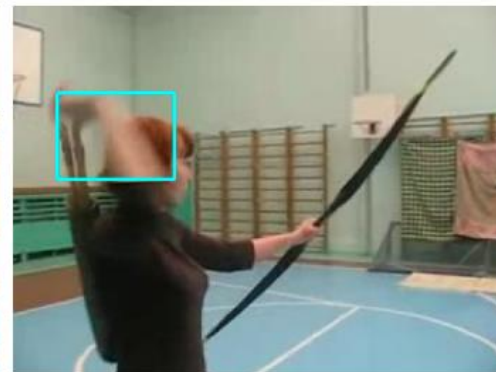
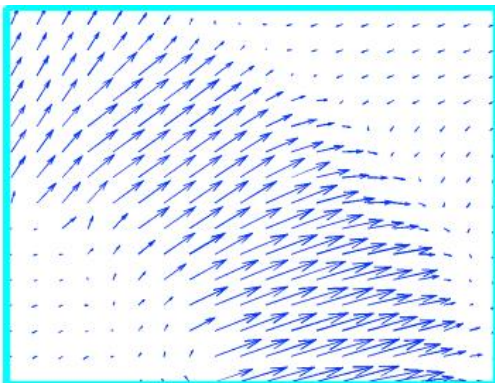
- motion patterns
- how things move over time

Good for: *“how things are moving”*

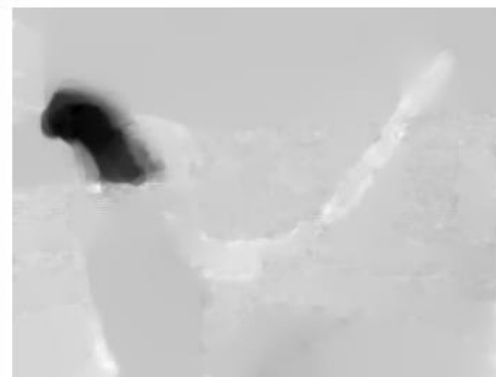


# Two-Stream CNN: Optical Flow

- Arrows show **direction + magnitude of motion**
- Longer arrows = faster movement
- Direction = where pixels are going

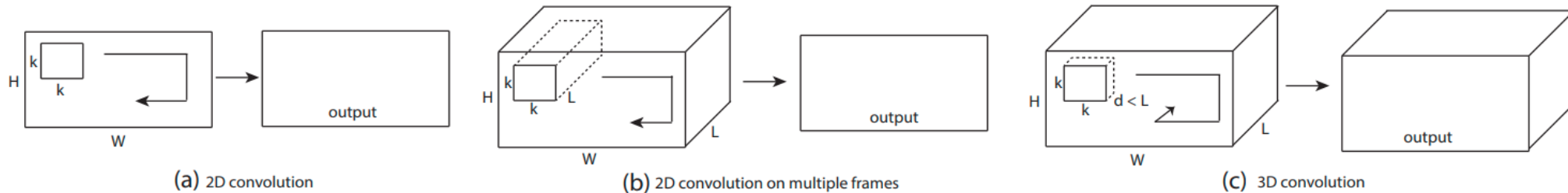


- Grayscale image of **horizontal motion**
- Bright/dark = left/right movement



- Grayscale image of **vertical motion**
- Bright/dark = up/down movement

# C3D: Learning Spatiotemporal Features with 3D CNN



## 2D Convolution

- Standard CNN on **one image**
- Kernel moves over:
- height ( $H$ )
- width ( $W$ )

### Learns:

- spatial features only (edges, shapes, objects)

## 2D CNN on multiple frames

- Apply the same 2D CNN to **each frame separately**
- Then combine later

### Learns:

- spatial features per frame
- Still no true motion understanding (it doesn't connect frames directly)

## 3D Convolution

- Input is a **video clip**:

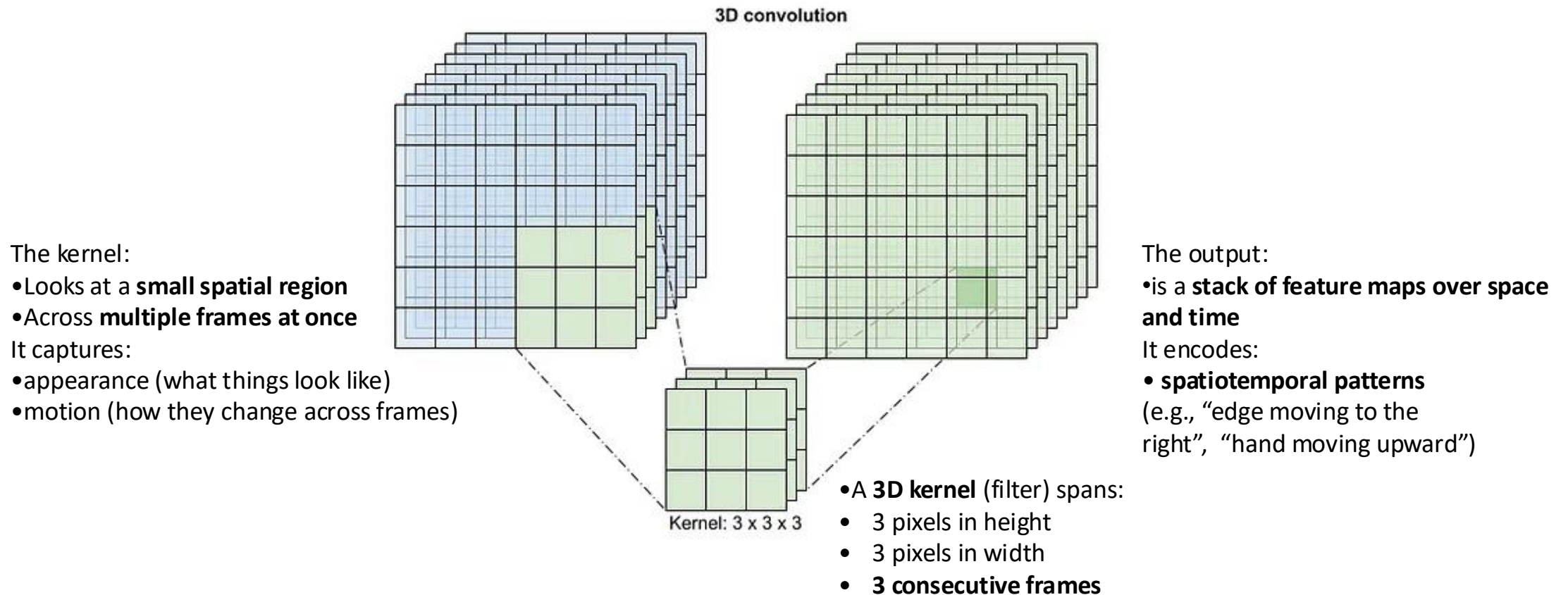
- height ( $H$ )
- width ( $W$ )
- time ( $L$  frames)

### Kernel is now 3D

### Learns:

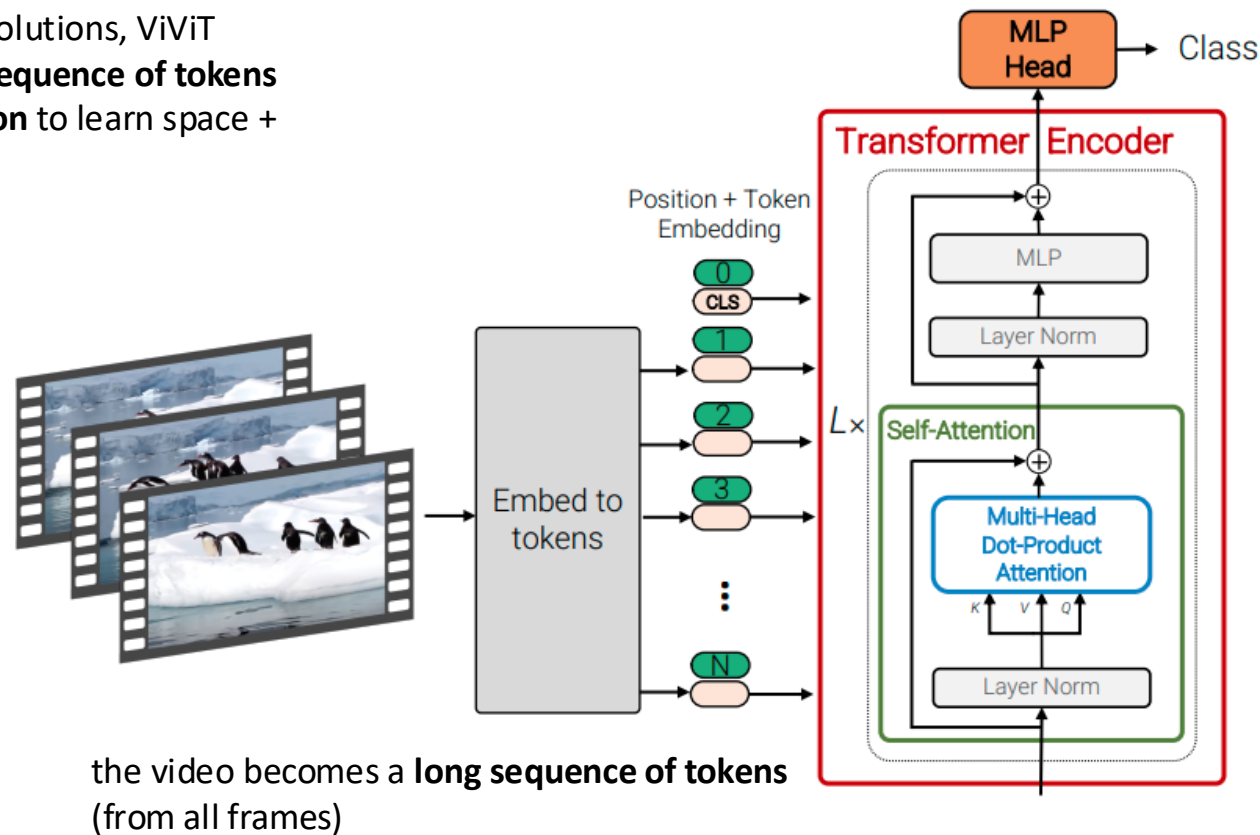
- appearance (like 2D CNN)
- motion (because it sees multiple frames together)

# C3D: Learning Spatiotemporal Features with 3D CNN



# ViViT: Video Vision Transformers

Instead of using convolutions, ViViT treats a video like a **sequence of tokens** and uses **self-attention** to learn space + time relationships

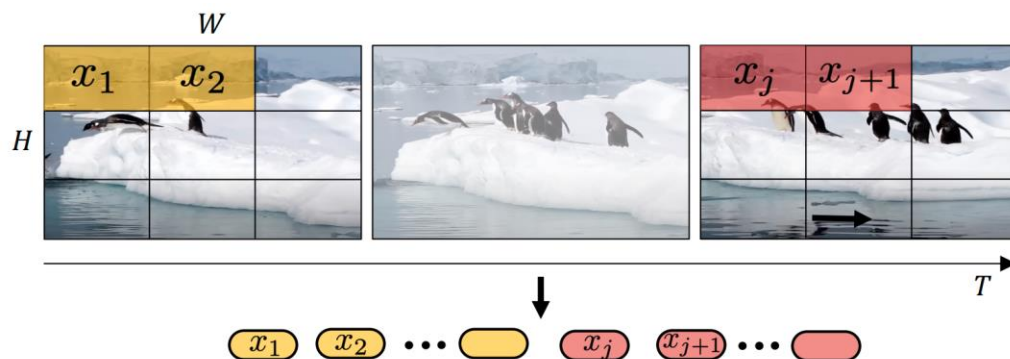


In **Self-Attention**:  
Each token can "look at" every other token  
So the model learns:

- spatial relations (within a frame)
- temporal relations (across frames)

# ViViT: Embeddings

A) Uniform Frame Sampling



B) Tubelet Embeddings

Instead of:

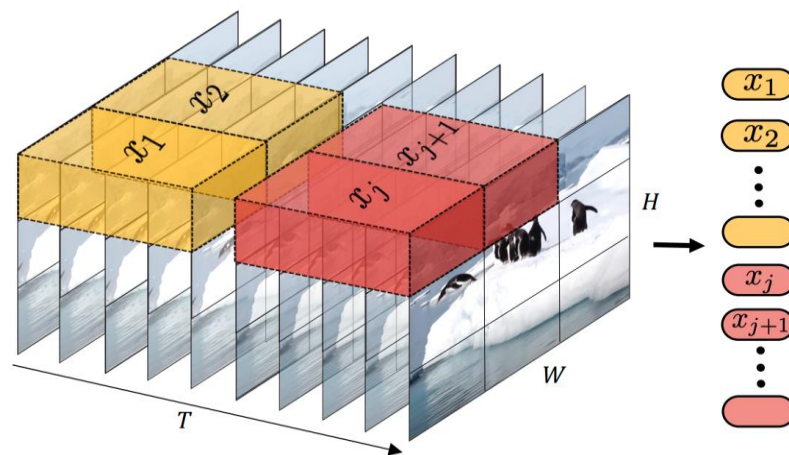
- 2D patches (like images)

We use:

**3D patches across space AND time**

A tubelet spans:

- height ( $H$ )
- width ( $W$ )
- time ( $T$ )



Each tubelet is converted into a single token, each representing a **spatiotemporal chunk**



# Exercises